Pattern–Use Case Mapping Matrix

IndustryApp – post-accident repair offer platform · PA1

Authors: Ömer Efe Öztürk, Bahadır İhsan Herdem · **Course:** BIL 482 Design Patterns · **Version:** 1.0 · **Date:** 05/10/2026

Table of Contents

1. Document Task Matrix
2. Pattern–Use Case Mapping Matrix
3. Explanations and Justifications per Use Case
    a. UC-01: Role-Based Registration and Login
    b. UC-02: Create Accident Report
    c. UC-03: Receive Job Notification and Submit Offer
    d. UC-04: Select Offer and Reveal Contact Information
4. System-Wide Patterns
5. Teaching Notes
6. Coverage Check

1. Document Task Matrix

#	Task in creating this document	Responsible	Reviewer
T1	Mapping matrix (codes, classes, use cases)	Bahadır İhsan Herdem	Ömer Efe Öztürk
T2	UC-01 and UC-04 pattern justifications	Ömer Efe Öztürk	Bahadır İhsan Herdem
T3	UC-02 and UC-03 pattern justifications	Bahadır İhsan Herdem	Ömer Efe Öztürk
T4	System-wide patterns and teaching notes	Ömer Efe Öztürk	Bahadır İhsan Herdem
T5	Coverage check against the assignment	Bahadır İhsan Herdem	Ömer Efe Öztürk
T6	Final review, formatting, GitHub upload	Bahadır İhsan Herdem	Ömer Efe Öztürk

2. Pattern–Use Case Mapping Matrix

Sixteen patterns are planned in 18 places: 16 belong to a use case and 2 apply to the whole system. Every use case has at least two patterns, above the minimum of one. Each `PAT` code will appear unchanged in the code comment, the design document and the test that will verify it.

Code	Pattern	Type	Use case	Main classes	Measurable effect
PAT-01.1	Factory Method	Creational	UC-01	`IUserProfileFactory`, `CustomerProfileFactory`, `MechanicProfileFactory`	Adding a role changes 0 existing classes
PAT-01.2	Abstract Factory	Creational	UC-01	`IDashboardFactory`, `Customer/Mechanic/AdminDashboardFactory`	Wrong role–panel combinations in tests: 0
PAT-02.1	Builder	Creational	UC-02	`AccidentReportBuilder`	Incomplete reports rejected: 100%
PAT-02.2	Composite	Structural	UC-02	`IDamageComponent`, `DamageCategory`	New category = data only, 0 code changes
PAT-02.3	Decorator (image pipeline)	Structural	UC-02	`ResizeProcessor`, `CompressionProcessor`, `ExifStripProcessor`	GPS tags in stored photos: 0
PAT-02.4	Adapter (file storage)	Structural	UC-02	`IFileStorage` ← `LocalFileStorage`	Application lines changed when storage changes: 0
PAT-03.1	Observer / Publish–Subscribe	Behavioral / architectural	UC-03 (reused in all)	`IEventDispatcher`, `IEventHandler<T>`, `InProcessEventDispatcher`	Report creation p95 < 500 ms, independent of matching
PAT-03.2	Strategy (matching)	Behavioral	UC-03	`IMatchingStrategy`, `CityMatchingStrategy`, `RadiusMatchingStrategy`	Adding a strategy changes 0 classes
PAT-03.3	Chain of Responsibility (+ Template Method)	Behavioral	UC-03	`IMechanicFilter`, `MechanicFilterBase`, 4 filters	Adding a filter breaks 0 existing filter tests
PAT-03.4	Decorator (price)	Structural	UC-03	`IPriceComponent`, `BasePrice`, `PartsCost`, `PaintCost`, `WarrantyCost`, `OtherCost`	Total = sum of lines (property test)
PAT-03.5	Adapter (notification channels)	Structural	UC-03	`INotificationChannel` ← `SmtpEmailChannel`, `LoggingSmsChannel`	New SMS provider = 1 class
PAT-03.6	Circuit Breaker + Retry	Resilience	UC-03	`ResilientNotificationChannel` (Polly)	Constant processing time while the provider is down; `CircuitOpen` records

Code	Pattern	Type	Use case	Main classes	Measurable effect
PAT-04.1	State	Behavioral	UC-04	`IReportState`, 5 state classes, `ReportStateFactory`	Invalid transitions rejected: 100%
PAT-04.2	Command	Behavioral	UC-04	`IReportCommand`, `SelectOfferCommand`, `IReportCommandFactory`	State after undo = state before selection
PAT-04.3	Proxy (protection)	Structural	UC-04	`IContactInfo`, `ContactInfo`, `ContactInfoProxy`	Real numbers in unauthorized responses: 0
PAT-04.4	Strategy (sorting)	Behavioral	UC-04	`IOfferSortStrategy` + 3 sort classes	New sort order = 1 class
PAT-X.1	Repository + Unit of Work	Architectural	All	`I*Repository`, `IUnitOfWork`	Unit tests running without a database: 100%
PAT-X.2	Singleton (DI container)	Creational	All	`IDateTimeProvider`, event queue, strategy resolvers	Static `Instance` fields in code: 0

3. Explanations and Justifications per Use Case

For each pattern: why it fits the use case, how it will be applied in the architecture, and what it improves in flexibility, modularity or maintainability.

3.1 UC-01: Role-Based Registration and Login

PAT-01.1 Factory Method

- **Why:** customer and mechanic registration share one flow (create user → profile → role → consents) but build different profiles and require different legal documents. Without the pattern, `AuthService.RegisterAsync` would grow an if/switch per role.
- **How:** `IUserProfileFactory` declares `Role`, `RequiredLegalDocumentCodes` and `CreateProfileAsync`. `CustomerProfileFactory` and `MechanicProfileFactory` (Application layer) implement it; `AuthService` picks the factory whose `Role` matches the request and lets it create the profile.
- **Benefit:** a new role is one new factory class registered in DI; the registration service and its tests do not change (open–closed).

PAT-01.2 Abstract Factory

- **Why:** after login each role needs a menu, a summary and quick actions that must belong together; a customer must never receive a mechanic menu.
- **How:** `IDashboardFactory` creates the three parts; `CustomerDashboardFactory`, `MechanicDashboardFactory` and `AdminDashboardFactory` are the families. `AccountService.GetDashboardAsync` selects one factory by role and builds all parts

from it; the React layout draws its menu from this response.

- **Benefit:** mixing families is structurally impossible; adding a role panel is one family, and the frontend needs no role-specific menu code.

3.2 UC-02: Create Accident Report

PAT-02.1 Builder

- **Why:** a report is collected over five wizard steps and is valid only with at least one photo, one damage item, a location and a vehicle type. A constructor with many parameters or a half-filled entity could reach the database.
- **How:** `AccidentReportBuilder` (Domain) offers `WithCustomer`, `WithVehicleType`, `AddDamage`, `AddPhoto`, `AtLocation`, `WithDescription` and `Build(now, ttl)`; `Build` throws `REPORT_INCOMPLETE` if a rule fails and otherwise returns an Open report that raises `AccidentReportCreatedEvent`.
- **Benefit:** the validity rule lives in one place and is unit-tested; new optional fields are new builder methods without touching callers.

PAT-02.2 Composite

- **Why:** damage categories form a three-level tree (e.g. Body → Front → Bumper) that the wizard, matching (root categories) and the admin screen all walk.
- **How:** `IDamageComponent` exposes `Children`, `IsLeaf` and a recursive `Contains(categoryId)`; `DamageCategory` implements it for both branches and leaves. The frontend tree and admin management use the same shape.
- **Benefit:** clients treat a single category and a whole subtree the same way; a new level or category is data only.

PAT-02.3 Decorator (image pipeline)

- **Why:** every photo must be resized, compressed and stripped of EXIF/GPS, and steps may be added or reordered later (e.g. watermark).
- **How:** `ImageSharpProcessor` is the base component that loads the image; `ResizeProcessor`, `CompressionProcessor` and `ExifStripProcessor` each wrap an `IImageProcessor` and add one step. The chain is composed in DI (Infrastructure).
- **Benefit:** each step is a small, separately tested class; changing the pipeline means changing the composition, not a large method.

PAT-02.4 Adapter (file storage)

- **Why:** the use case must not depend on where files live (server disk today, S3 or MinIO later).

- **How:** Application defines `IFileStorage` (`SaveAsync` , `OpenReadAsync` , `MoveAsync` , `DeleteAsync`); `LocalFileStorage` adapts the .NET file system API to it and adds path-traversal protection.
- **Benefit:** switching storage is one new adapter class with zero changes in Application.

3.3 UC-03: Receive Job Notification and Submit Offer

PAT-03.1 Observer / Publish–Subscribe

- **Why:** report creation must not know about matching, and matching must not know about e-mail; direct calls would chain services together and make report creation wait for them.
- **How:** entities raise domain events; after commit, `InProcessEventDispatcher` queues them and a background service runs every `IEventHandler<T>` (e.g. `AccidentReportCreatedMatchingHandler` , `MechanicsMatchedNotificationHandler`). The same mechanism serves UC-04 notifications.
- **Benefit:** modules are loosely coupled; a new reaction is a new handler with no change to the publisher, and the HTTP response time is independent of matching.

PAT-03.2 Strategy (matching)

- **Why:** how candidates are found (same city, or within a radius) is a policy that should change by configuration.
- **How:** `IMatchingStrategy` with `CityMatchingStrategy` (`CITY`) and `RadiusMatchingStrategy` (`RADIUS` , Haversine distance); `MatchingStrategyResolver` picks the one named in `Matching:Strategy` and fails at start-up for an unknown code.
- **Benefit:** a new matching policy is one class; no if/else inside `MatchingService` .

PAT-03.3 Chain of Responsibility (+ Template Method)

- **Why:** candidates pass several independent checks (approved and active, specialty, availability, daily capacity) whose order may change.
- **How:** each filter extends `MechanicFilterBase` , whose template method fixes the skeleton (filter → stop if empty → pass to next) and leaves `FilterAsync` to subclasses. `MechanicFilterChainBuilder` links the filters in the order of `Matching:Filters` ; each step logs how many candidates it removed.
- **Benefit:** filters are tested in isolation, can be reordered in configuration, and a new filter does not affect existing ones.

PAT-03.4 Decorator (price)

- **Why:** an offer price is labor plus optional parts, paint, warranty and other items, and the customer must see the breakdown, not one number.

- **How:** `BasePrice` (labor) is wrapped by `PartsCost`, `PaintCost`, `WarrantyCost` and `OtherCost`, each adding its amount to `Total` and a line to `Lines`. `Offer.Create` takes the resulting `IPriceComponent` and stores its total and lines.
- **Benefit:** the total always equals the sum of the lines; a new price item type is one decorator class.

PAT-03.5 Adapter (notification channels)

- **Why:** notification logic must not depend on MailKit or a future SMS provider's API.
- **How:** `INotificationChannel` is implemented by `SmtpEmailChannel` (MailKit, HTML templates) and `LoggingSmsChannel` (masked log, no provider); `NotificationService` calls the channels listed for each notification.
- **Benefit:** adding an SMS provider is one adapter class; services stay free of protocol details.

PAT-03.6 Circuit Breaker + Retry

- **Why:** if the e-mail provider is slow or down, every notification would wait for a timeout and the event queue would stall.
- **How:** `ResilientNotificationChannel` wraps a channel with a Polly pipeline (exponential retry, circuit breaker on failure ratio, timeout), configured from `Resilience:Email` and registered as a singleton so the circuit state is shared. When the circuit is open, the delivery is recorded as `CircuitOpen` and in-app delivery continues.
- **Benefit:** a failing external service cannot slow the system; behavior is tuned in configuration and visible in `notification_deliveries`.

3.4 UC-04: Select Offer and Reveal Contact Information

PAT-04.1 State

- **Why:** what may happen to a report depends on its status (Open, OfferSelected, Completed, Cancelled, Expired); status checks spread across services would be easy to miss.
- **How:** `IReportState` declares every operation; `ReportStateBase` rejects all by default and each state class overrides only what it allows (e.g. `OpenState` accepts select, cancel, expire). `AccidentReport` delegates each operation to `ReportStateFactory.For(Status)`; only state classes can call the internal `ApplyTransition`.
- **Benefit:** allowed transitions are explicit and testable per class; a new status is a new class instead of edits across the code.

PAT-04.2 Command

- **Why:** selecting an offer changes the report and several offers together, and it must be reversible within 10 minutes.
- **How:** `SelectOfferCommand` holds the report, the chosen offer and the other pending offers; `Execute()` applies the selection through the State, `Undo()` reverses it, `CreateSnapshot()` saves the affected offer ids in the status history. `IReportCommandFactory` rebuilds the command from the snapshot when the customer undoes.
- **Benefit:** execute and undo are symmetric and kept in one object, undo logic is not duplicated, and every selection leaves an audit record.

PAT-04.3 Proxy (protection)

- **Why:** the decision to show a phone number must be made in exactly one place; if every screen checked it, one could forget.
- **How:** `ContactInfoProxy` implements `IContactInfo` around the real `ContactInfo` and the report; it returns the real phone and e-mail only when `report.IsMatchedWith(viewerId)` is true, otherwise a masked number and `-`. `ContactService` always returns the proxy.
- **Benefit:** access control is centralized and unit-tested; new screens get the protection automatically.

PAT-04.4 Strategy (sorting)

- **Why:** the customer sorts offers by price, fastest completion or newest, and more orders may follow (e.g. by rating).
- **How:** `IOfferSortStrategy` with `PriceAscendingSort`, `FastestCompletionSort` and `NewestFirstSort`; `OfferService` selects the strategy by the request's sort key, defaulting to price.
- **Benefit:** a new order is one class; the controller and service contain no sorting switch.

4. System-Wide Patterns

PAT-X.1 Repository + Unit of Work

- **Why:** use cases must not depend on EF Core, and one use case must save several aggregates in one transaction (e.g. report + offers + history in UC-04).
- **How:** Application defines one `I<Entity>Repository` per aggregate and `IUnitOfWork`; Infrastructure implements them. `UnitOfWork.SaveChangesAsync` commits, translates unique and concurrency violations into 409 conflicts, and only then hands the collected domain events to the dispatcher.
- **Benefit:** all unit tests run without a database; persistence technology can change without touching services.

PAT-X.2 Singleton (through the DI container)

- **Why:** some objects must exist once per application (the clock, the event queue, strategy resolvers, the resilience pipeline whose circuit state must be shared).
- **How:** they are registered with singleton lifetime in the container instead of a static `Instance` property.
- **Benefit:** one shared instance without global state; every singleton can be replaced by a fake in tests.

5. Teaching Notes

- **Proxy vs. Decorator:** both implement the same interface and wrap an inner object. `ContactInfoProxy` controls access and adds no behavior; `PriceComponentDecorator` adds behavior and restricts nothing. `ResilientNotificationChannel` is structurally a Decorator but its intent is Circuit Breaker.
- **Strategy vs. Chain of Responsibility:** in UC-03 the Strategy answers *how do we find candidates*, the chain answers *which candidates pass*. One strategy runs; every filter in the chain runs in order.
- **State + Command:** the Command packages a reversible multi-step change; the State decides whether each step is allowed now. Together they give an undoable status transition with an audit trail.
- **Publish–Subscribe vs. Circuit Breaker:** publish–subscribe manages the business flow between modules; the circuit breaker protects system health when an external service fails.

6. Coverage Check

The assignment requires at least one pattern per use case, each named, justified and described in the architecture; this matrix meets that with a margin.

- **UC-01:** 2 patterns — Factory Method, Abstract Factory.
- **UC-02:** 4 patterns — Builder, Composite, Decorator (images), Adapter (storage).
- **UC-03:** 6 patterns — Observer / Publish–Subscribe, Strategy, Chain of Responsibility with Template Method, Decorator (price), Adapter (channels), Circuit Breaker + Retry.
- **UC-04:** 4 patterns — State, Command, Proxy, Strategy (sorting); Observer is reused for its notifications.
- **All use cases:** Repository + Unit of Work, Singleton via DI.

Each pattern above is (1) named, (2) justified with the problem it solves in its use case, and (3) placed in a specific layer and class of the architecture, with its benefit for flexibility, modularity or maintainability and a measurable effect that will be checked by a test. The verifying tests will be listed in the Software Design Document (PA2/PA3); planned examples are TC-04.3.1 `Undo_WithinWindow_RestoresPendingOffers` for PAT-04.2 and TC-03.5.1 `ResilientChannel_OpensCircuitAfterFailures` for PAT-03.6.